# JAVASCRIPT

## NOTIONS AVANCÉES



No notes on this slide.

# JAVASCRIPT

## Les modules

Il est bien de découper son code en plusieurs fichiers. Par exemple, on pourra définir la classe `Disk` dans son propre fichier, avec également une constante globale pour le cercle unitaire en (0;0).

```
// Fichier Disk.js
class Disk{
    constructor(x, y, radius){
        this.x = x;
        this.y = y;
        this.radius = radius;
    }
    area(){
        return Math.PI*this.radius*this.radius;
    }
}

const UNIT_DISK = new Disk(0, 0, 1);
```

Comment utiliser ce code dans un autre fichier ?

No notes on this slide.

# JAVASCRIPT

## Les modules

Il serait possible d'importer tous les fichiers dans le HTML principal, à l'aide de balises `<script>`, mais cela a plusieurs inconvénients, en particulier

1. Tout est importé globalement, généralement des conflits de nom. Ainsi, dans l'exemple précédent, il ne peut y avoir un autre objet ou variable se nommant `Disk`.
2. Si une librairie est composée de plusieurs fichiers, ils doivent tous être importés un par un. Cela devient encombrant avec des centaines de fichiers.
3. La gestion des dépendances est très manuelle, complexe, et sujette aux erreurs.

Il existe une solution à tout ces problèmes : **Les modules**



Les modules fournissent aussi, comme on va le voir plus loin, une notion d'encapsulation.

# JAVASCRIPT

## Les modules

Un module est un code contenu dans un fichier qui s'exécute dans un contexte isolé. Tout ce qui n'est pas exporté d'un module ne sera pas visible depuis l'extérieur.

Pour exporter un élément, on utilise le mot clé `export`.

```
// Fichier Disk.js
export class Disk{                              // Classe exportée
    constructor(x, y, radius){
        this.x = x;
        this.y = y;
        this.radius = radius;
    }
    area(){
        return Math.PI*this.radius*this.radius;
    }
}

export const UNIT_DISK_AREA = new Disk(0, 0, 1).area(); // Constante exportée
const UNIT_DISK_PERIMETER = 2*Math.PI;                  // Constante non exportée
```

No notes on this slide.

# JAVASCRIPT

## Les modules

Pour importer un élément d'un module, on utilisera le mot-clé `import`

```
// Fichier main.js
import { Disk, UNIT_DISK_AREA } from "./Disk.js"

const myDisk = new Disk(2, 3, 1.5);

if(myDisk.area() > UNIT_DISK_AREA) {
    console.log("myDisk plus grand que le cercle unité");
}

import { UNIT_DISK_PERIMETER } from "./Disk.js" // Erreur ! Car non exporté
```

Les imports seront indiqués entre `{` et `}`, même si un seul élément est importé

On ne peut importer des éléments que dans un module. En utilisant ce principe tout notre code sera un ensemble de modules.



Il est courant de faire du re-export, c'est-à-dire d'importer des éléments dans un module et de les y exporter. Cela permet à un utilisateur de ne pas faire trop d'imports différents. Par exemple, si vous créez une librairie avec un module "Button.js" et un module "Slider.js", vous pouvez créer un module "Widget.js" qui importe puis réexporte Button et Slider. En utilisant votre librairie il suffira de faire "import {Button, Slider} from 'Widget.js'

# JAVASCRIPT

## Les modules

Pour chaque module, il est possible d'indiquer un export par défaut

```
// Fichier Disk.js
export default class Disk{ ... } // Export par défaut
export const UNIT_DISK = new Disk(0, 0, 1);
```

L'export par défaut peut être importé en lui donnant n'importe quel nom, sans accolades. JavaScript sait qu'un import sans accolades est forcément l'export par défaut du module.

```
// Fichier main.js
import Disk from "./Disk.js" // Valide
const disk = new Disk(1,2,3);
```

```
// Fichier main.js
import Cercle from "./Disk.js" // Également valide, ici Disk est appelé Cercle
const disk = new Cercle(1,2,3);
```



No notes on this slide.

# JAVASCRIPT

## Les modules

Il est possible de renommer un import avec le mot-clé `as`. Cela est particulièrement utile quand on importe deux éléments dont les noms sont les mêmes.

```
import { Disk as Cercle, UNIT_DISK as CERCLE_UNITE} from "./Disk.js"
```

C'est typiquement le cas quand on utilise plusieurs librairies différentes dans un seul fichier

```
import { Color as ColorLib1} from "./my/library1/Color.js"
import { Color as ColorLib2} from "./my/library2/Color.js"
```



No notes on this slide.

# JAVASCRIPT

## Les modules

Il est possible de mélanger les imports et les imports par défaut

```
// Importe l'import pas défaut sous le nom de "Cercle",
// et l'import "UNIT_DISK" sous le nom de "CERCLE_UNITE"
import Cercle, {UNIT_DISK as CERCLE_UNITE} from "./Disk.js"
```

Attention à ne pas abuser du renommage. A utiliser seulement pour éviter des collisions de noms ou éviter des ambiguïtés, sinon la compréhension du code devient très difficile.

No notes on this slide.

# JAVASCRIPT

## Les modules

Pour importer un module JavaScript dans du HTML, on utilise la balise `<script>` avec `type="module"`

```
<script type="module">
    import Disk from "./Disk.js";
    /* ... */
</script>
```

Attention : l'isolation vaut toujours dans ce cas, et le contenu de la balise `script` sera isolé du reste du code

```
<script>
    const value = 10;
</script>
<script type="module">
    console.log(value); // value n'existe pas ici, affichera undefined
</script>
<script>
    console.log(value); // affichera 10
</script>
```



No notes on this slide.

# JAVASCRIPT

## Les modules - Références

Site officiel : https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

W3Schools: https://www.w3schools.com/js/js_modules.asp



No notes on this slide.